

🚀 IPC 统御引擎独立运行与开发操作手册 (IPC Engine Operation Manual)

本手册专为 **IPC 计划系统开发者与高级计划员** 编写，旨在指导如何独立进行 **C++ 统御引擎内核** 的编译、命令行启动运行、数据同步萃取、核心算法运算、数据库回写持久化以及底层源码阅读与调试。

🔗 一、引擎架构与全流程逻辑 (Engine Core Architecture)

IPC 统御引擎采用了 **内存与物理数据库彻底解耦** 的高性能 SoA (Structure of Arrays) 架构。整体数据处理分为 “**萃取 - 计算 - 回写**” 三大标准阶段：

```
graph TD
    DB["DB[(DuckDB 物理数据库: ipc.db / ipc_stress.db)]"]
    
    subgraph Phase_1 ["Phase 1: ECL 萃取同步 (DB -> C++ Memory)"]
        Adapter["Adapter[src/database.cpp: DbAdapter]"]
        DB -->|SQL 批量查询| Adapter
        Adapter -->|解析为 C++ SoA 内存向量| PartVocab["PartVocab[PartVocab 词表与物料节点]"]
    end
    
    subgraph Phase_2 ["Phase 2: C++ 裸金属计算引擎 (In-Memory Solver)"]
        IBP["IBP[IBP 需求拆解与 Holt-Winters 安全库存]"]
        LSC["LSC[LSC 多维 BOM 动态展开树]"]
        LLC["LLC[LLC 拓扑底层码编译]"]
        MEIO["MEIO[MEIO 多阶安全库存优化]"]
        MRP["MRP[LBL MRP 净需求消纳与一/二/三类替代料分配]"]
        Pegging["Pegging[供需 Pegging 匹配账本生成]"]
        DBD["DBD[MCDS DBD 有限产能微观派程引擎]"]
        Coproduct["Coproduct[联副产品配方与产出规划]"]
    end
    
    PartVocab --> IBP --> LSC --> LLC --> MEIO --> MRP --> Pegging --> DBD --> Coproduct
    Coproduct --> DB
```

```

subgraph Phase 3: 流式持久化回写 (C++ Memory -> DB)
  Appender[duckdb::Appender 极速流式写入]
  Coproduct --> Appender
  DBD --> Appender
  Pegging --> Appender
  MRP --> Appender
  Appender --> |物理事务提交与表覆盖/更新| DB
end

```

三阶段关键操作流：

1. DB 萃取 (Sync From DB) : `ipc::DbAdapter` (基于 [src/database.cpp](#)) 通过 SQL 将 `ipc_material_node` (物料)、`ipc_onhand` (库存)、`ipc_scheduled_receipt` (在途/在制)、`ipc_bom_item` (BOM 结构与替代优先级)、`ipc_independent_demand` (独立需求)、`ipc_allotment_constraint` (战术配额) 等加载入 C++ 内存结构。

2. 内存求解 (C++ Compute Engine) :

- 依赖项与低层码 (LLC) 拓扑排序；
- IBP 需求比例拆解与 Holt-Winters 统计预测；
- MEIO 多阶安全库存传导计算；
- LBL MRP 经典水位消纳与一/二/三类替代料按优先级动态分配；
- MCDS DBD 时空网格有限产能微观派程；
- 联副产品分级降级规划。

3. 数据库持久化 (Sync Back To DB) : 利用 DuckDB 极速 C API (`duckdb::Appender`) 批量回写写入 `ipc_planned_order` (计划订单)、`ipc_supply_assignment` / `ipc_planned_supply_assignment` (供需 Pegging 匹配)、`ipc_swap_result` / `ipc_alternate_allocation` (替代料日志)、`ipc_dispatch_ledger` (派程明细) 等结果表。

二、环境准备与编译构建 (Compilation & Build)

1. 编译环境要求

- 操作系统 : Windows 10 / 11 / Windows Server 64-bit
- 编译器 : MSVC C++20 (Visual Studio 2022 / 2019 Build Tools, 需支持 `cl.exe`)
- 依赖库 : DuckDB C/C++ 头文件与动态库 (`duckdb.dll` & `IPC/lib/duckdb.lib`)

2. 一键编译主引擎

打开 **VS Developer Command Prompt (vcvars64.bat)** 或者直接在终端执行根目录的编译批处理 :

```
compile_main.bat
```

编译产物 :

- `main_mem3.exe` : IPC C++ 裸金属统御引擎主执行程序。
- `test_runner_validation.exe` : 供 Python 控制塔/Streamlit 大盘调用的重算可执行程序。
- `build/*.obj` : 中间对象文件 (存放在 `build/` 目录中, 保证根目录干净)。

三、引擎命令行独立运行指南 (CLI Execution)

开发者或计划员可以直接在命令行窗口独立拉起 `main_mem3.exe` , 支持灵活的命令行参数控制 :

1. 命令行参数说明

参数项	默认值	可选值	说明
<code>--db <path></code>	<code>`data/ipc.db`</code>	任意 DuckDB 文件路径 (如 <code>`ipc.db`</code> , <code>`data/ipc_stress.db`</code>)	指定引擎连接并读取/回写的物理数据库文件

参数项	默认值	可选值	说明
`--mode <mode>`	`iop`	`iop` / `itp` / `mrp`	指定求解器模式：`iop` (滚动操作排产), `itp` (战术配额平衡), `mrp` (经典 MRP 消纳)
`--step <step>`	`all`	`all` / `lbl` / `dbd`	指定计算步骤：`all` (全流程计算), `lbl` (仅材料 LBL MRP 展开), `dbd` (仅能力派程)
`--scenario <id>`	`baseline`	字符串 (如 `sandbox_01`)	指定规划场景 ID，用于战术配额与沙箱推演
`--repl`	*无*	*标志位*	计算完成后**启动交互式 SQL 命令行**，可直接在终端查账
`--stress`	*无*	*标志位*	运行引擎极限压力测试套件（200 万 SKU 级别）

2. 常用运行示例指令

场景 1：标准全流程 IOP 排程与数据库持久化

```
main_mem3.exe --db ipc.db --mode iop --step all
```

场景 2：计算完成后开启 REPL 交互式 SQL 查账

```
main_mem3.exe --db ipc.db --mode iop --repl
```

在 ``grits_ipc_db>`` 提示符下可直接输入 SQL，例如：

```
SELECT main_part, alt_part, allocated_qty, day FROM ipc_alternate_allocation LIMIT 10;
EXIT
```

场景 3：战术配额 ITP 模式运算（针对特定的沙箱场景）

```
main_mem3.exe --db ipc.db --mode itp --scenario sandbox_scenario_a
```

场景 4：仅执行材料 LBL MRP 净需求消纳（跳过能力派程）

```
main_mem3.exe --db ipc.db --step lbl
```

四、核心代码结构与阅读指南 (Code Navigation)

如果需要深入调试或修改引擎逻辑，请参考以下核心源码目录映射：

h:/IPC/	
├─ include/	# 头文件定义
│ └─ ipc_types.h	# 【核心】全局数据结构定义 (PartSiteRecord, FlatBomItem, PlannedOrder 等)
├─ ipc/	# 模块接口声明 (namespace ipc)
│ └─ database.h	# DbAdapter 数据库适配器头文件
│ └─ mrp_engine.h	# LBL MRP 净需求与替代料计算头文件
│ └─ dbd_engine.h	# MCDS DBD 有限产能派程头文件
│ └─ coproduct.h	# 联副产品维度规划头文件
│ └─ lsc_tree.h	# 多维 LSC BOM 展开树头文件
│ └─ vocab.h	# PartVocab 词表与 LLC 拓扑头文件

└─ math_utils.h	# Holt-Winters 与统计函数头文件
└─ src/	# C++ 核心算法实现
└─ engine_main.cpp	# 【主入口】main() 函数，解析 CLI、调度各阶段算法、
Appender 数据回写	
└─ database.cpp	# DbAdapter 提取算法：从 DuckDB 读取数据构建 C++ SoA
└─ mrp_engine.cpp	# LBL-MRP 水位消纳算法实现与 1/2/3 类替代料决策
└─ dbd_engine.cpp	# MCDS DBD 时空网格派程算法与 ATP 回溯
└─ coproduct.cpp	# 联副产品分级降级算法
└─ lsc_tree.cpp	# 多维条件 BOM 展开网络编译
└─ vocab.cpp	# 字符串词表映射与全局 LLC 低层码计算
└─ substitution.cpp	# 替代料业务规则与配额分配逻辑
└─ math_utils.cpp	# Holt-Winters 预测与 Normal Inverse CDF 实现
└─ tests/	# 测试套件源码
└─ holographic_stress_test.cpp	# 46 项微观算子单元测试
└─ itp_iop_stress_test.cpp	# 200 万级压力测试源码

调试与修改关注点：

1. 修改/新增 SQL 数据字段：在 [src/database.cpp](#) 中调整 DbAdapter 加载逻辑，并在 [include/ipc types.h](#) 中增加对应 C++ 结构体字段。
2. 调整 MRP 替代料逻辑：定位至 [src/mrp_engine.cpp](#) 及 [src/substitution.cpp](#)。
3. 调整产能派程算法：定位至 [src/dbd engine.cpp](#)。
4. 数据库回写与表结构升级：定位至 [src/engine_main.cpp](#) 的 Phase H（同步对账数据灌入部分）。

 五、数据库对账与结果校验 (Database Inspection)

引擎运行完成后，计算结果已流式持久化至 DuckDB 数据库中。您可以通过以下核心物理表进行对账分析：

数据库表名	描述	关键字段
-------	----	------

数据库表名	描述	关键字段
`ipc_planned_order`	**计划订单表** (生成的建议补货订单)	`ipc_planned_order`, `part`, `qty`, `request_start_date`, `due_date`
`ipc_planned_supply_assignment`	**计划补货 Pegging 匹配表**	`demand`, `part`, `supply`, `assigned_qty`, `due_date`
`ipc_supply_assignment`	**在途/现货 Pegging 匹配表**	`demand`, `part`, `supply`, `supply_type`, `assigned_qty`
`ipc_alternate_allocation`	**替代料分配日志表**	`main_part`, `alt_part`, `allocated_qty`, `day`, `alt_class`
`ipc_swap_result`	**不完全替代换货日志表**	`demand_code`, `from_part`, `to_part`, `swapped_qty`, `swap_reason`
`ipc_dispatch_ledger`	**DBD 微观派程账本**	`part_code`, `order_qty`, `scheduled_day`, `allocated_capacity`
`ipc_allotment_ledger`	**战术配额缺口账本**	`scenario_id`, `part_code`, `allotment_limit`, `consumed_qty`, `blocked_demand_qty`
`ipc_material_node`	**物料节点主数据与安全库存**	`part`, `site`, `safety_stock`, `selling_ave_price`

快速查账 SQL 示例：

```
-- 1. 查看算出的前 20 条计划补货订单
SELECT part, qty, request_start_date, due_date FROM ipc_planned_order LIMIT 20;

-- 2. 统计各替代料的使用情况
SELECT main_part, alt_part, SUM(allocated_qty) as total_alt_qty
FROM ipc_alternate_allocation
GROUP BY main_part, alt_part;

-- 3. 查看供需 Pegging 匹配关系
SELECT demand, part, supply, supply_type, assigned_qty
FROM ipc_planned_supply_assignment
LIMIT 20;
```

🌐 六、与 Python 控制塔集成调用 (Integration with Cockpit/Server)

当前端可视化控制塔 (<http://127.0.0.1:8501>) 中点击 “**执行自上而下分解与排产拉动**” 时，Python FastAPI 服务 ([server.py](#)) 会在后台自动拉起 `test_runner_validation.exe` (或 `main_mem3.exe`)：

```
# Python server.py 中的后台调用示意
subprocess.run([
    "test_runner_validation.exe",
    "--db", db_path,
    "--mode", solver_mode,
    "--scenario", scenario_id
], check=True)
```

如需启动完整控制塔系统，可运行：

```
start_cockpit.bat
```

或者运行：

```
run_all.bat
```

💡 总结 (Quick Recap)

1. 想改代码 -> 查看 [src/engine_main.cpp](#) 与 `include/ipc/` 下对应的头文件。
2. 想重新编译 -> 在命令行执行 `compile_main.bat` 。
3. 想手动跑引擎 -> 执行 `main_mem3.exe --db ipc.db --mode iop --repl` 。
4. 想看算出的结果 -> 在 DBeaver 中打开 `ipc.db` 或在 `--repl` 交互终端中查询 SQL !